AUTO JUDGE REPORT

Introduction:

A machine learning solution developed to automate the assessment of programming problem complexity. As the volume of competitive programming content grows, manual difficulty tagging becomes inconsistent and time-consuming. This project bridges that gap by providing a data-driven approach to categorize problems and predict fine-grained difficulty scores.

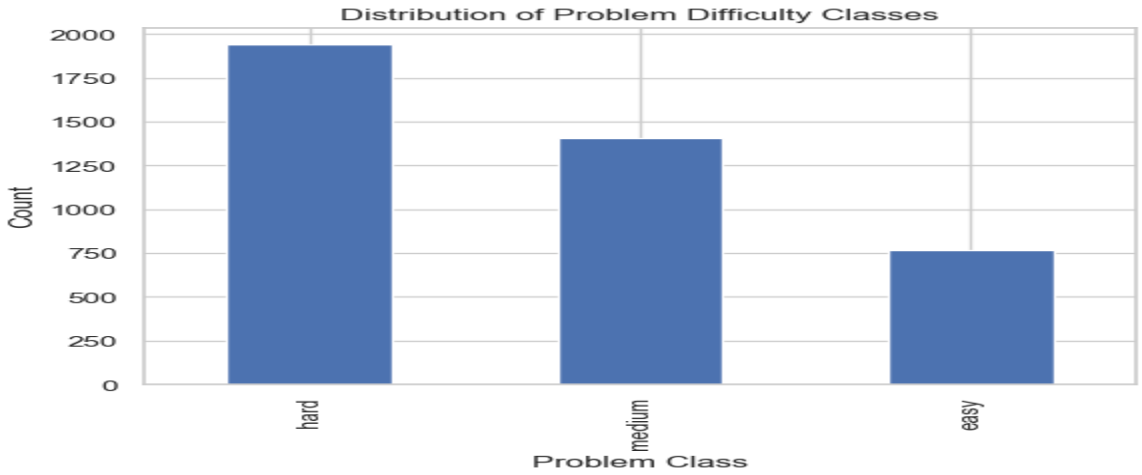
Objectives:

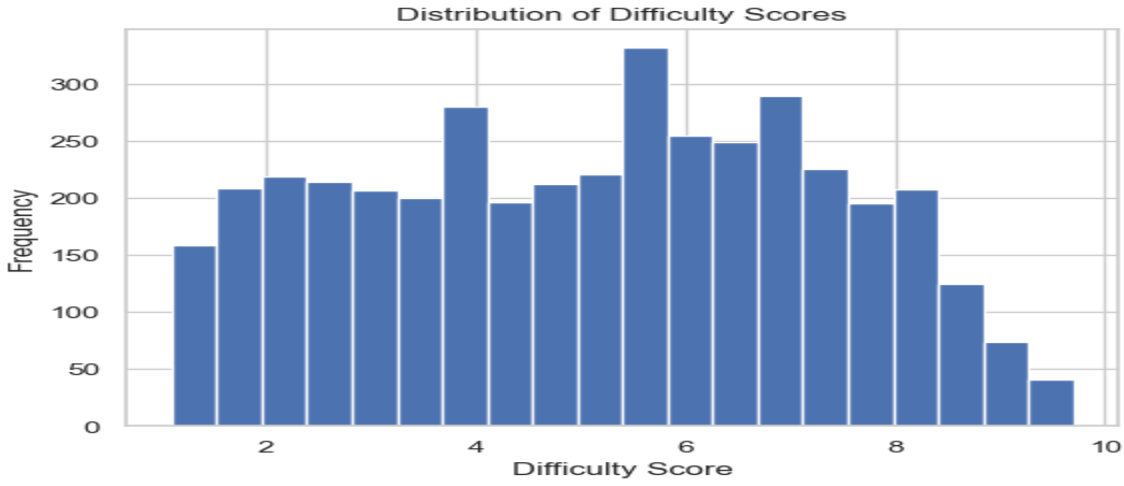
- Develop a Classification Model to categorize problems into "Easy," "Medium," or "Hard."
- Develop a Regression Model to provide a numerical difficulty score.
- Deploy an interactive Web Interface for real-time inference.

Exploratory Data Analysis (EDA):

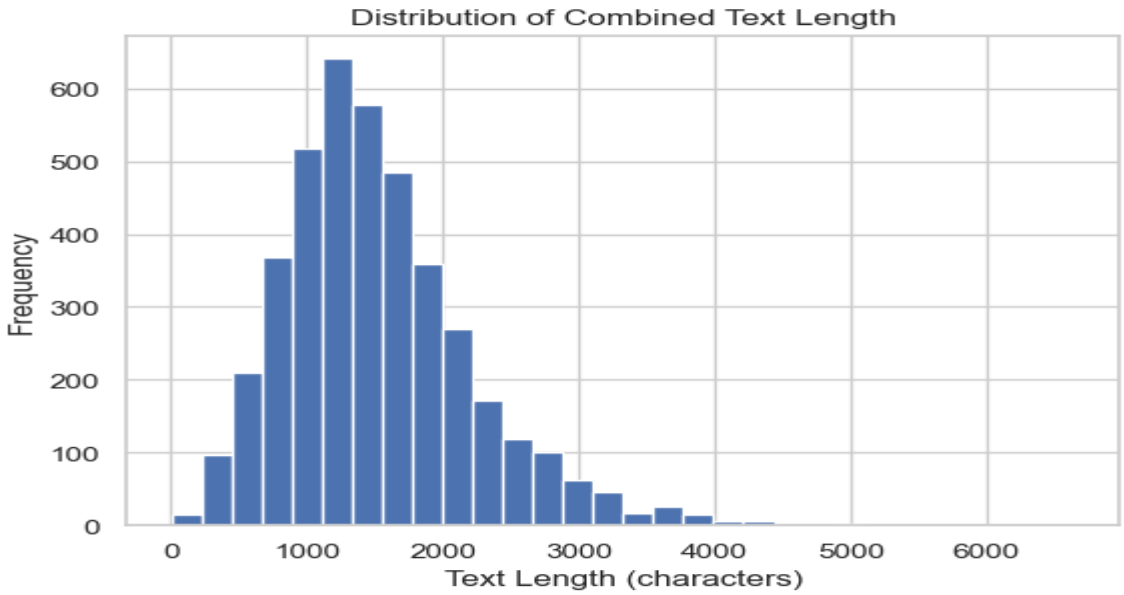
Before modeling, a thorough analysis of the dataset was conducted to understand feature distributions and correlations.

- Difficulty Distribution: Analyzed the balance between Easy, Medium, and Hard classes. It was observed that "Hard" problems often have distinct linguistic patterns.

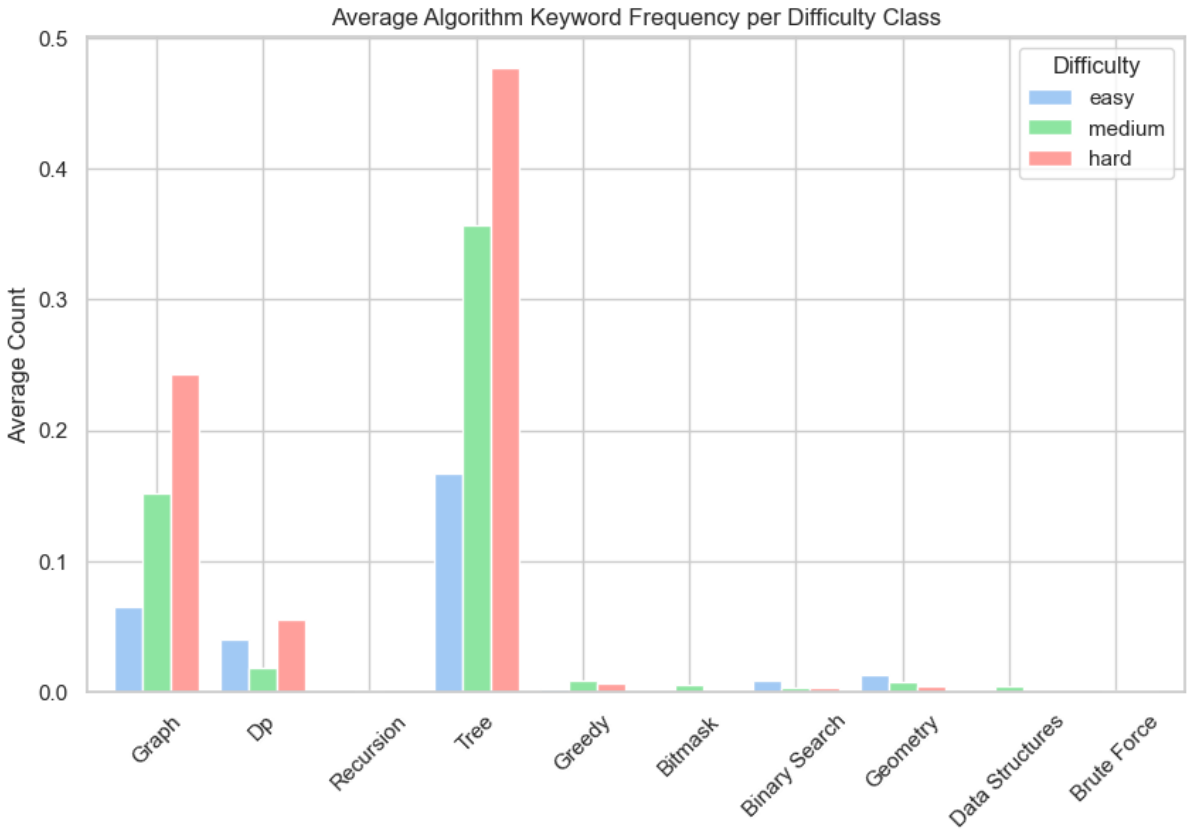




- Text Length Analysis: Discovered a correlation between description length and problem complexity; Hard problems tend to have more detailed constraints and explanations.



- Keyword Frequency: Visualized the occurrence of specific algorithmic terms (e.g., "Dynamic Programming," "Dijkstra," "Bitmask").



- Symbol Density: Identified that the frequency of mathematical symbols ($+, -, \le, \ge$) increases significantly with problem difficulty.

Preprocessing & Feature Engineering:

- TF-IDF Vectorization: Used to extract semantic meaning from titles and descriptions, focusing on the top 2,000 unigrams and bigrams.
- Manual Feature Extraction: Engineered three specific types of features:
    - Text Metrics: Character and word counts.
    - Logic Indicators: Counts of mathematical operators.
    - Algorithmic Keywords: Binary and frequency counters for question tags of problem.
- Data Splitting: A standard 80/20 train-test split was used to ensure unbiased evaluation.

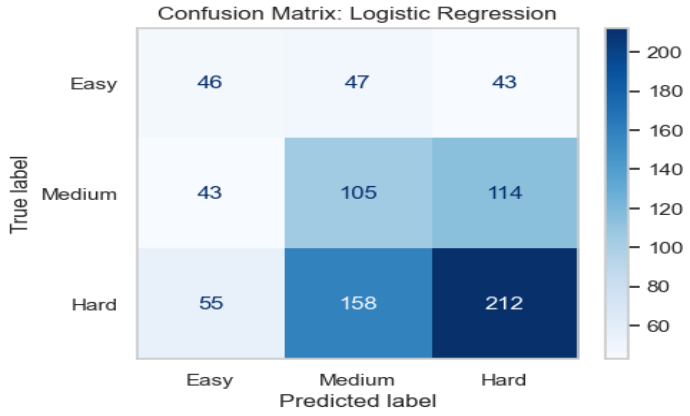
Model selection:

Classification:

1)Logistic regression

Final Accuracy: 0.4411

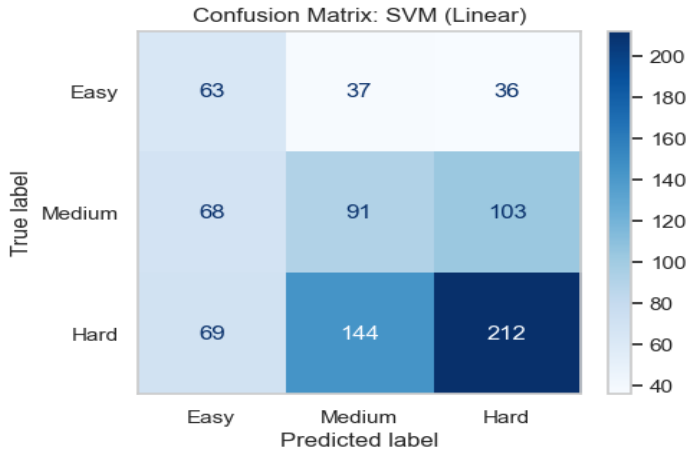
Macro F1 Score: 0.4099



2)SVM classification:

Final Accuracy: 0.4447

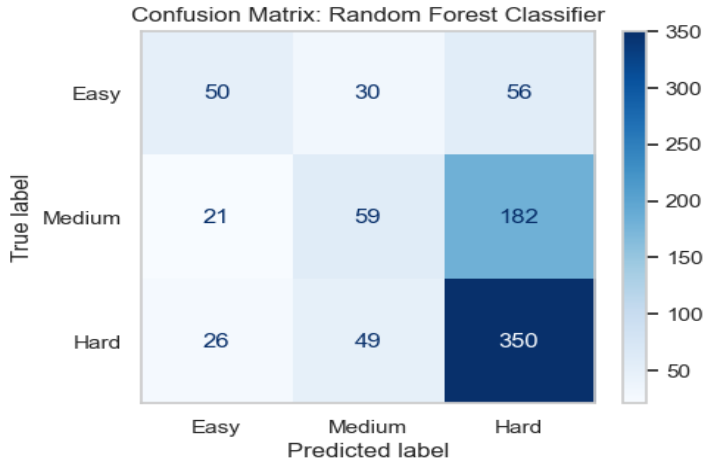
Macro F1 Score: 0.4207



3)Random forest classifier:

Final Accuracy: 0.5577

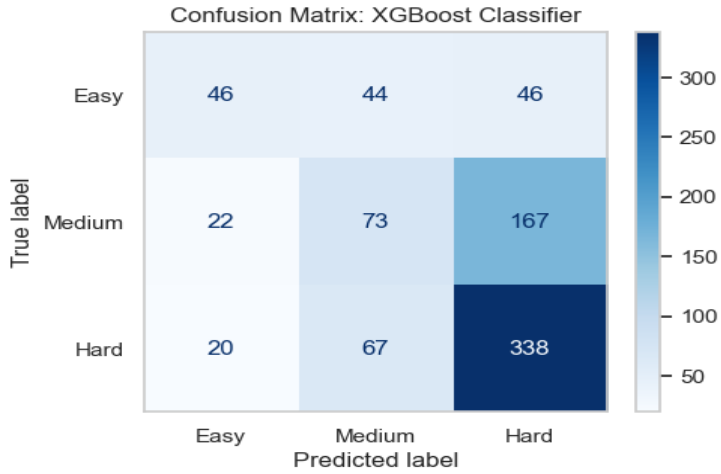
Macro F1 Score: 0.4717



4)XGBoost classifier:

Final Accuracy: 0.5553

Macro F1 Score: 0.4769



Conclusion :

From the above observations Random forest classifier preformed best from the remaining models with an accuracy of **55.77%.**

So we choose Random forest classifier as the base model for the classification task**.**

Regression model:

1)Linear regression:

Evaluation metrics:

Mean absolute error : 2.2981

Root mean square error : 2.8809.

2)Ridge regression:

Evaluation metrics:

Mean absolute error : 2.4314

Root mean square error : 3.0655

3)Random forest regressor:

Evaluation metrics:

Mean absolute error : 1.6862

Root mean square error : 2.0382

4)XGboost regressor:

Evaluation metrics:

Mean absolute error : 1.7014

Root mean square error : 2.0416

Conclusion:

From the above metrics Random forest regressor gave the best results with a mean absolute error of **1.6862** and root mean square error of **2.0382.**

So we choose Random forest regressor for regression task to find the difficulty score of the problem.

Save final model:

Save the trained model in .pkl formate in the same folder for building the web application.

UI & Web Implementation:

To make the model accessible, a professional web application was developed using Streamlit.

- Functionality: Users can input a problem's title, description, and I/O format.
- Real-time Inference: The app loads serialized joblib models to provide instant predictions.
- User Experience: Includes a visual dashboard showing the predicted class and a metric-style display for the difficulty score.

Vedio link for project demo:

https://drive.google.com/drive/folders/1WhkEIMqHTLkZ1ze-z47YvZ7iXg2EG9dv?usp=sharing

Conclusion:

The project successfully demonstrates that textual patterns and structural heuristics can be used to quantify algorithmic complexity. While linear models provided a strong baseline, the non-linear decision boundaries of the Random Forest model captured the nuance of competitive programming problems more accurately.

Future Work: Future iterations could incorporate Transformer-based models (BERT/RoBERTa) for deeper semantic understanding and utilize Regular Expressions to explicitly parse time and memory constraints as features.

Thanking you,

B.N.S. PAVAN KUMAR,

23116023,

ECE.